1. Upload the dataset

```
from google.colab import files
uploaded = files.upload()  # Select day.csv
```

```
Choose Files day.csv
day.csv(text/csv) - 57569 bytes, last modified: 11/4/2026 - 100% done
Saving day.csv to day (2).csv
```

2. Import libraries and load data

```
import io
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

sns.set_theme(style="whitegrid")
```

```
# Load the uploaded daily dataset
file_name = next((name for name in uploaded if name.lower().endswith(".csv")), None)
if file_name is None:
    raise ValueError("day.csv")

df = pd.read_csv(io.BytesIO(uploaded[file_name]))
print("Dataset shape:", df.shape)
display(df.head())
df.info()
```

```
Dataset shape: (731, 16)
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casua
0	1	2011-01-01	1	0	1	0	6	0	2	0.344167	0.363625	0.805833	0.160446	33
1	2	2011-01-02	1	0	1	0	0	0	2	0.363478	0.353739	0.696087	0.248539	13
2	3	2011-01-03	1	0	1	0	1	1	1	0.196364	0.189405	0.437273	0.248309	12
3	4	2011-01-04	1	0	1	0	2	1	1	0.200000	0.212122	0.590435	0.160296	10
4	5	2011-01-05	1	0	1	0	3	1	1	0.226957	0.229270	0.436957	0.186900	8

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 731 entries, 0 to 730
Data columns (total 16 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   instant     731 non-null    int64
 1   dteday      731 non-null    object
 2   season      731 non-null    int64
 3   yr          731 non-null    int64
 4   mnth        731 non-null    int64
 5   holiday     731 non-null    int64
 6   weekday     731 non-null    int64
 7   workingday  731 non-null    int64
 8   weathersit  731 non-null    int64
 9   temp        731 non-null    float64
 10  atemp       731 non-null    float64
 11  hum         731 non-null    float64
 12  windspeed   731 non-null    float64
```

3. Data quality inspection

```
print("Missing values by column:")
display(df.isna().sum().to_frame("missing_values"))

print("Duplicate rows:", df.duplicated().sum())
print("Date range:", df["dteday"].min(), "to", df["dteday"].max())
print("Target summary:")
display(df["cnt"].describe())
```

```
Missing values by column:
```

	missing_values
instant	0
dteday	0
season	0
yr	0
mnth	0
holiday	0
weekday	0
workingday	0
weathersit	0
temp	0
atemp	0
hum	0
windspeed	0
casual	0
registered	0
cnt	0

```
Duplicate rows: 0
Date range: 2011-01-01 to 2012-12-31
Target summary:
```

	cnt
count	731.000000
mean	4504.348837
std	1937.211452
min	22.000000
25%	3152.000000
50%	4548.000000
75%	5956.000000
max	8714.000000

dtype: float64

4. Data cleaning and feature engineering

```
# Work on a copy and parse the date
data = df.copy()
data["dteday"] = pd.to_datetime(data["dteday"], errors="coerce")

# Remove rows with invalid dates or missing target values, if any
data = data.dropna(subset=["dteday", "cnt"]).copy()
data = data.drop_duplicates().sort_values("dteday").reset_index(drop=True)

# Calendar features derived from the date
data["date_year"] = data["dteday"].dt.year
data["date_month"] = data["dteday"].dt.month
data["date_day"] = data["dteday"].dt.day
data["date_dayofweek"] = data["dteday"].dt.dayofweek
data["date_dayofyear"] = data["dteday"].dt.dayofyear
```

```
# Target is total rental count
target = "cnt"

# Exclude identifiers, raw date, and casual/registered to prevent leakage:
# cnt is the sum of casual + registered.
feature_cols = [
    "season", "yr", "mnth", "holiday", "weekday", "workingday",
    "weathersit", "temp", "atemp", "hum", "windspeed",
    "date_month", "date_day", "date_dayofweek", "date_dayofyear"
]

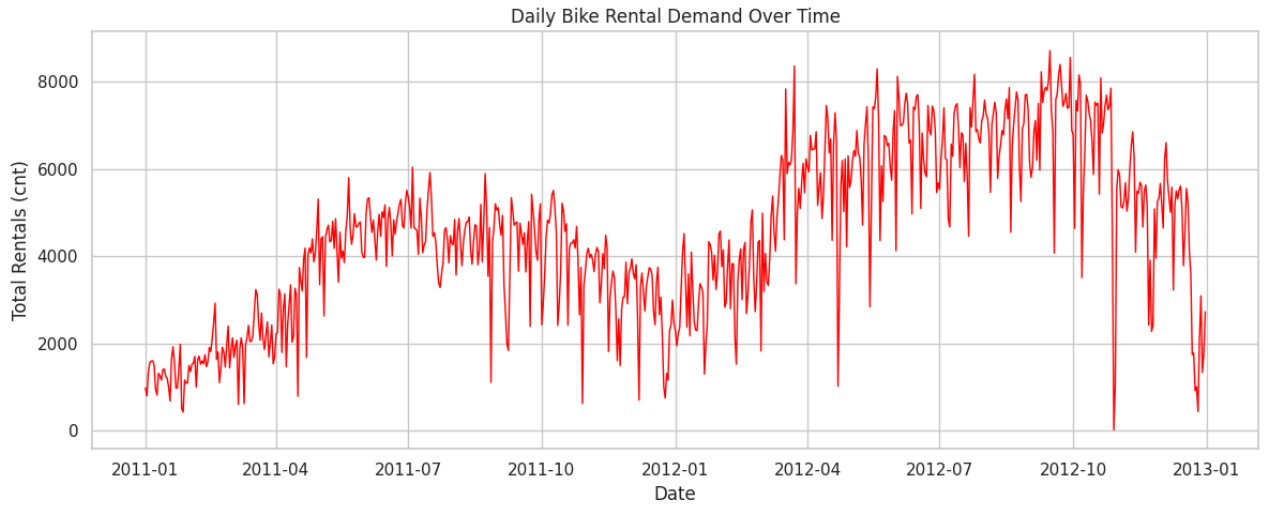
X = data[feature_cols].copy()
y = data[target].astype(float)

print("Cleaned data shape:", data.shape)
print("Predictor count:", len(feature_cols))
print("Target:", target)
```

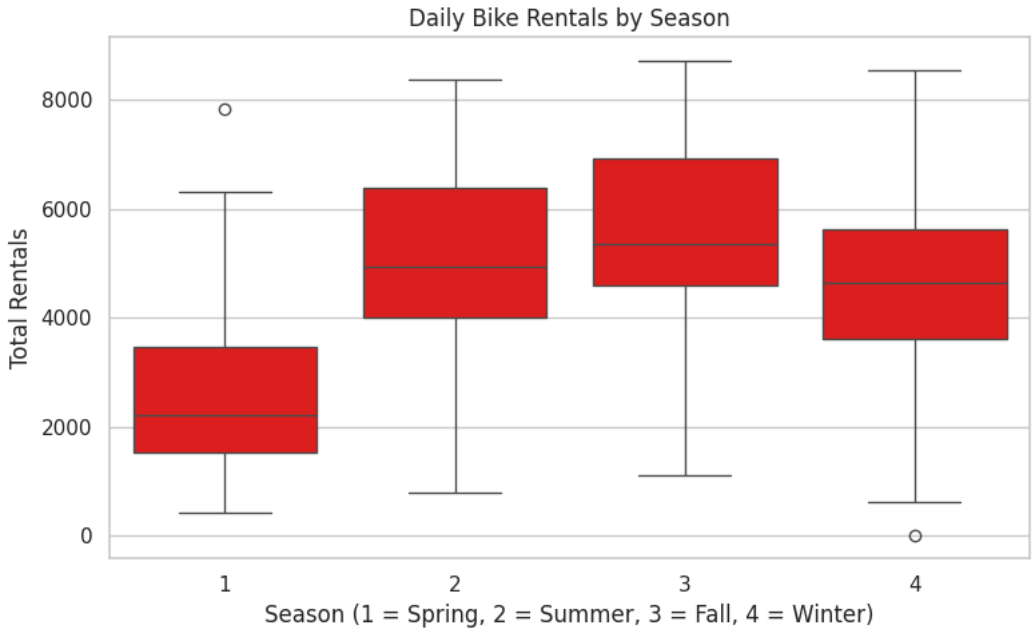
```
Cleaned data shape: (731, 21)
Predictor count: 15
Target: cnt
```

5. Exploratory data analysis

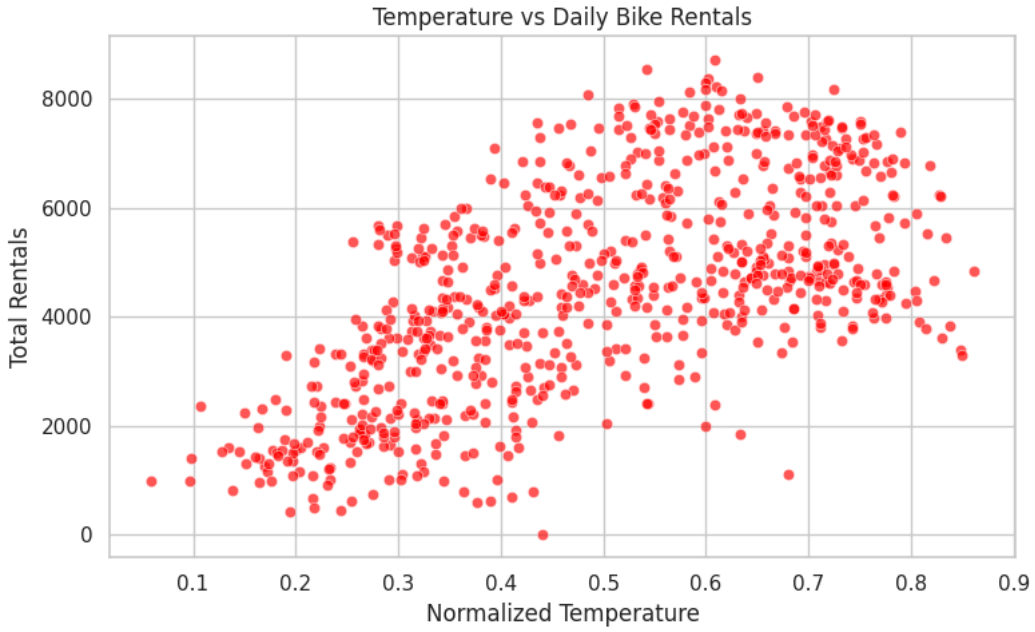
```
plt.figure(figsize=(12, 5))
plt.plot(data["dteday"], data["cnt"], linewidth=1, color="red")
plt.title("Daily Bike Rental Demand Over Time")
plt.xlabel("Date")
plt.ylabel("Total Rentals (cnt)")
plt.tight_layout()
plt.show()
```



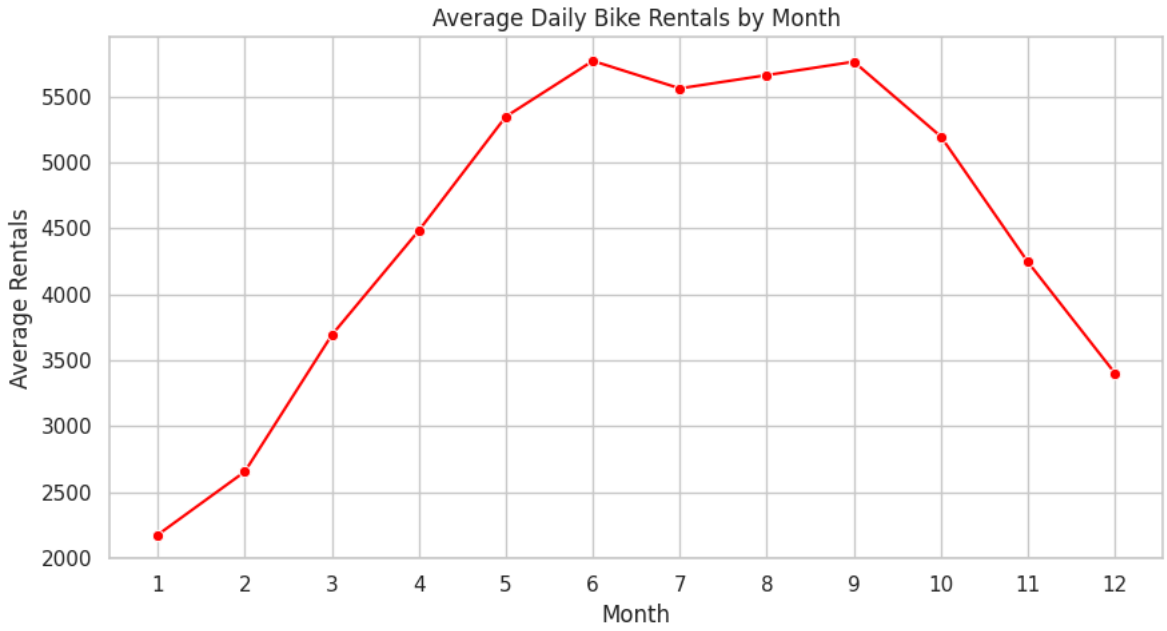

```
plt.figure(figsize=(8, 5))
sns.boxplot(data=data, x="season", y="cnt", color="red")
plt.title("Daily Bike Rentals by Season")
plt.xlabel("Season (1 = Spring, 2 = Summer, 3 = Fall, 4 = Winter)")
plt.ylabel("Total Rentals")
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=data, x="temp", y="cnt", alpha=0.65, color="red")
plt.title("Temperature vs Daily Bike Rentals")
plt.xlabel("Normalized Temperature")
plt.ylabel("Total Rentals")
plt.tight_layout()
plt.show()
```




```
monthly = data.groupby("mnth", as_index=False)["cnt"].mean()
plt.figure(figsize=(9, 5))
sns.lineplot(data=monthly, x="mnth", y="cnt", marker="o", color="red")
plt.title("Average Daily Bike Rentals by Month")
plt.xlabel("Month")
plt.ylabel("Average Rentals")
plt.xticks(range(1, 13))
plt.tight_layout()
plt.show()
```



6. Chronological train-test split

```
# Use the earliest 80% of dates for training and the latest 20% for testing.
split_index = int(len(data) * 0.80)

X_train, X_test = X.iloc[:split_index], X.iloc[split_index:]
y_train, y_test = y.iloc[:split_index], y.iloc[split_index:]
test_dates = data["dteday"].iloc[split_index:]

print("Training period:", data["dteday"].iloc[0].date(), "to", data["dteday"].iloc[split_index - 1].date())
print("Testing period:", data["dteday"].iloc[split_index].date(), "to", data["dteday"].iloc[-1].date())
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

```
Training period: 2011-01-01 to 2012-08-06
Testing period: 2012-08-07 to 2012-12-31
Training records: 584
Testing records: 147
```

7. Build and compare prediction models

```
categorical_features = [
    "season", "yr", "mnth", "holiday", "weekday", "workingday", "weathersit",
    "date_month", "date_day", "date_dayofweek"
]
numeric_features = ["temp", "atemp", "hum", "windspeed", "date_dayofyear"]

# Linear Regression pipeline: impute, scale numeric variables, one-hot encode categories.
linear_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler())
        ]), numeric_features),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]), categorical_features)
    ]
)

linear_model = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LinearRegression())
])

# Random Forest pipeline: impute numeric and categorical columns, then fit trees.
rf_preprocessor = ColumnTransformer(
    transformers=[
        ("num", SimpleImputer(strategy="median"), numeric_features),
```

```
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]), categorical_features)
    ]
)

random_forest = Pipeline([
    ("preprocessor", rf_preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=400,
        max_depth=8,
        min_samples_leaf=3,
        random_state=42,
        n_jobs=-1
    ))
])

models = {
    "Linear Regression": linear_model,
    "Random Forest": random_forest
}

predictions = {}
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    pred = np.maximum(pred, 0)  # rental counts cannot be negative
    predictions[name] = pred

    mae = mean_absolute_error(y_test, pred)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)

    results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2
    })

results_df = pd.DataFrame(results).sort_values("RMSE").reset_index(drop=True)
display(results_df)
```

	Model	MAE	RMSE	R2
0	Linear Regression	789.974612	1091.140321	0.661207
1	Random Forest	917.909581	1128.031436	0.637911

Next steps: Generate code with `results_df` | New interactive sheet

8. Select the best model and evaluate

```
best_model_name = results_df.loc[0, "Model"]
best_predictions = predictions[best_model_name]

print("Best model by lowest test RMSE:", best_model_name)
print(f"MAE:  {mean_absolute_error(y_test, best_predictions):,.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, best_predictions)):,.2f}")
print(f"R²:   {r2_score(y_test, best_predictions):.3f}")
```

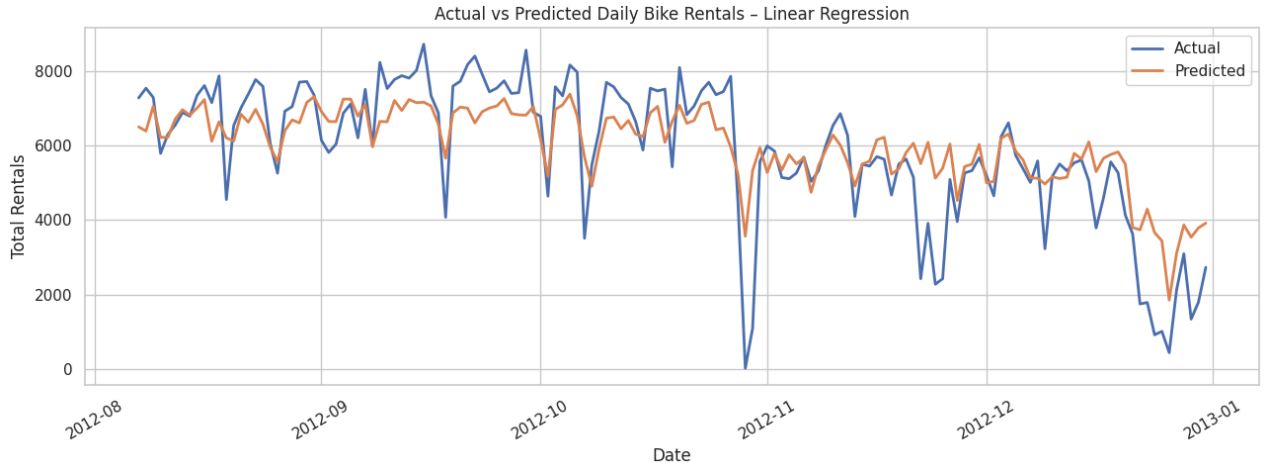
```
Best model by lowest test RMSE: Linear Regression
MAE:  789.97
RMSE: 1,091.14
R²:   0.661
```

9. Actual vs predicted visualization

```
comparison = pd.DataFrame({
    "Date": test_dates.values,
    "Actual": y_test.values,
    "Predicted": best_predictions
})
```

```
plt.figure(figsize=(13, 5))
plt.plot(comparison["Date"], comparison["Actual"], label="Actual", linewidth=2)
plt.plot(comparison["Date"], comparison["Predicted"], label="Predicted", linewidth=2)
plt.title(f"Actual vs Predicted Daily Bike Rentals – {best_model_name}")
plt.xlabel("Date")
plt.ylabel("Total Rentals")
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

display(comparison.head(10))
```



	Date	Actual	Predicted
0	2012-08-07	7273.0	6496.122185
1	2012-08-08	7534.0	6386.263727
2	2012-08-09	7286.0	7051.951313
3	2012-08-10	5786.0	6214.683668
4	2012-08-11	6299.0	6218.793780
5	2012-08-12	6544.0	6705.335667
6	2012-08-13	6883.0	6956.667719
7	2012-08-14	6784.0	6812.826699
8	2012-08-15	7347.0	7005.564428
9	2012-08-16	7605.0	7227.763704

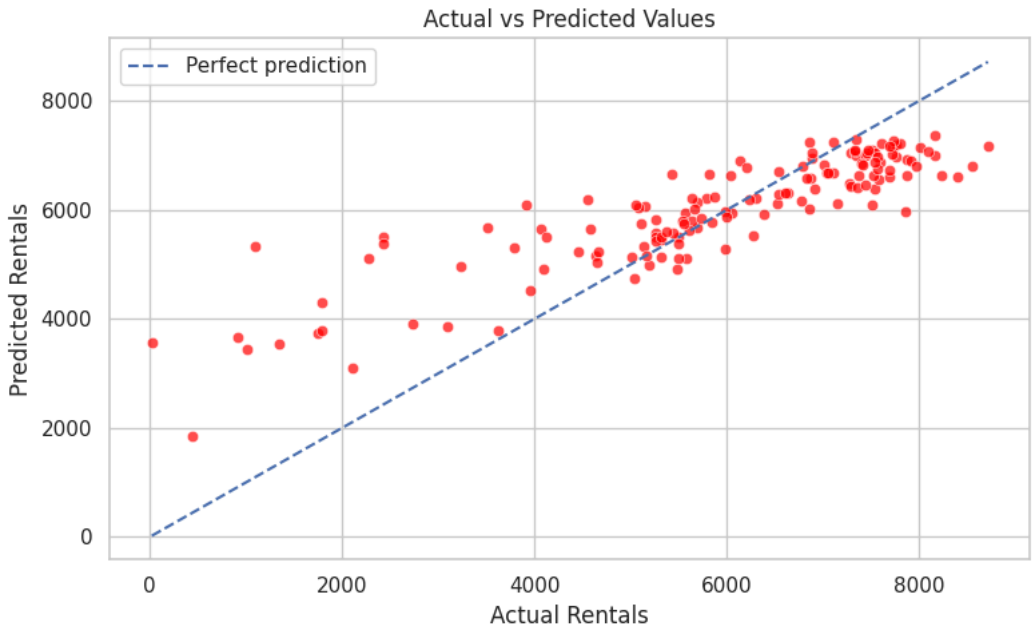
10. Prediction error analysis

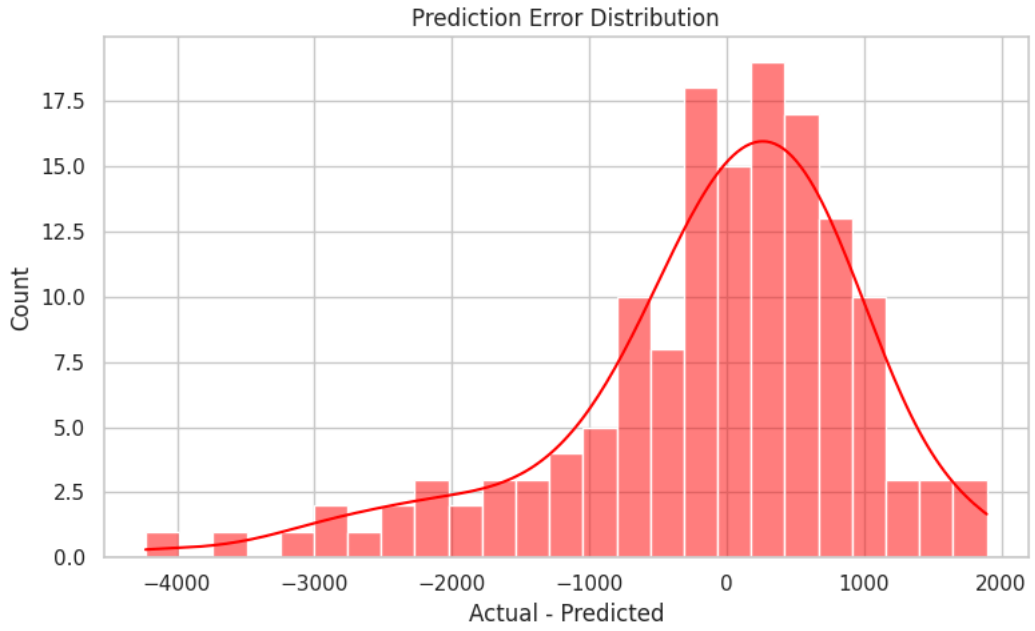
```
comparison["Error"] = comparison["Actual"] - comparison["Predicted"]

plt.figure(figsize=(8, 5))
sns.scatterplot(data=comparison, x="Actual", y="Predicted", alpha=0.7, color="red")
lims = [
    min(comparison["Actual"].min(), comparison["Predicted"].min()),
    max(comparison["Actual"].max(), comparison["Predicted"].max())
]
plt.plot(lims, lims, "--", label="Perfect prediction",)
plt.title("Actual vs Predicted Values")
plt.xlabel("Actual Rentals")
plt.ylabel("Predicted Rentals")
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
sns.histplot(comparison["Error"], bins=25, kde=True, color="red")
plt.title("Prediction Error Distribution")
plt.xlabel("Actual - Predicted")
plt.tight_layout()
```

```
plt.show()
```





11. Export results

```
# Save test predictions and model comparison for your project documentation.
comparison.to_csv("bike_rental_test_predictions.csv", index=False)
results_df.to_csv("model_evaluation_results.csv", index=False)

print("Saved:")
print("- bike_rental_test_predictions.csv")
print("- model_evaluation_results.csv")
```

```
Saved:
- bike_rental_test_predictions.csv
```